

AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy Part IV structure preview (draft)

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

Part IV — Occupancy Ground Rules

The memory ladder explains why warps stall. Occupancy asks: how many other warps do we need to fill those gaps?

The most fundamental rule of CUDA performance (book)

“Launch enough parallel work to fully occupy the GPU.”

Rule 1 — low occupancy, poor perf

First remedy: **increase parallelism** (more blocks/threads) until there are **enough ready warps to hide latency** — stop when performance **plateaus**.

- Up next: one operation ($C = A + B$, $N = 10^6$), two implementations, and what the profilers say about them.

*Plain English: rule 1 **fills the workshop with crews**; rule 2: a full workshop **still stalls** if the warehouse truck is the bottleneck.*

Rule 2 — occupancy already high

If the kernel is **memory-bound**, pushing to 100% won't help: you need **just enough warps to hide latency** — beyond that, the bottleneck is bandwidth.

Case Study, Take 1: addSequential

```
// Single thread does ALL N additions
__global__ void addSequential(const float* A,
    const float* B, float* C, int N) {
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        for (int i = 0; i < N; ++i) {
            C[i] = A[i] + B[i];
        }
    }
}
// Note: this kernel assumes <<<1,1>>>.
addSequential<<<1,1>>>>(d_A, d_B, d_C, N);
cudaDeviceSynchronize();
```

- One thread, one warp, one SM — **everything else watches.**
- Book: “the GPU’s vast resources are mostly idle... very poor occupancy and, ultimately, low performance.”
- No other warps to switch to $\Rightarrow$ memory waits become **pure idle time** — zero latency hiding.

Plain English: you rented the whole factory and hired one worker.

The Same Trap in PyTorch

```
import torch
N = 1_000_000
A = torch.arange(N, dtype=torch.float32,
                  device='cuda')
B = 2 * A
C = torch.empty_like(A)
torch.cuda.synchronize()
# Naive, sequential GPU ops - DO NOT DO THIS
with torch.inference_mode():
    for i in range(N): # N tiny serial kernels!
        C[i] = A[i] + B[i]
torch.cuda.synchronize()
```

- A Python loop around GPU ops **serializes N tiny kernel launches** — the GPU becomes “a scalar, nonparallel processor.”
- Same terrible occupancy, plus **per-launch CPU overhead** $\times$ 1,000,000.
- Book advice: there is **almost always an optimized PyTorch-native (vectorized) implementation** — including compiler-emitted code.

Plain English: the most common GPU bug: accidentally using the GPU as a very expensive single-core CPU.

Case Study, Take 2: addParallel — and $C = A + B$

```
// One thread per element
__global__ void addParallel(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C, int N) {
    int idx = blockIdx.x*blockDim.x + threadIdx.x;
    if (idx < N) C[idx] = A[idx] + B[idx];
}
addParallel<<<(N+255)/256, 256>>>(dA, dB, dC, N);
```

The book's full version also uses pinned host memory, a non-blocking stream, and `cudaMallocAsync/cudaMemcpyAsync` — the habits from Part II (full listing: [next page](#)).

Plain English: same factory, now every workstation is staffed — and the PyTorch version is just “hire the staffing agency.”

- $\sim 3,907$ blocks $\times$ 256 threads — **a million threads** cover the array (Figure 6-19).
- `__restrict__`: promises no pointer aliasing — frees the compiler to optimize.
- PyTorch equivalent is one line: $C = A + B$ — a single vectorized kernel engaging many threads concurrently.

addParallel, Full Version: The Part II Habits Assembled

```
cudaStream_t s;  
cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);  
float *h_A, *d_A; // same for B, C  
cudaMallocHost(&h_A, N * sizeof(float)); // pinned!  
cudaMallocAsync(&d_A, N * sizeof(float), s);  
cudaMemcpyAsync(d_A, h_A, N * sizeof(float),  
                cudaMemcpyHostToDevice, s);  
addParallel<<<<(N+255)/256, 256, 0, s>>>>(d_A,d_B,d_C,N);  
cudaMemcpyAsync(h_C, d_C, N * sizeof(float),  
                cudaMemcpyDeviceToHost, s);  
cudaStreamSynchronize(s); // wait for THIS stream only  
cudaFreeAsync(d_A, s); cudaStreamDestroy(s);
```

- Every line was taught in Part II: **pinned** host alloc (six-step flow); **non-blocking stream** + **MallocAsync/FreeAsync** (pools page).
- **cudaMemcpyAsync** is the new face: an asynchronous copy **enqueued into stream s** — it can overlap with compute **only because** the host buffer is pinned.
- **cudaStreamSynchronize(s)**: wait for this stream only — no global **cudaDeviceSynchronize**.

Plain English: same kernel, production-grade plumbing — every habit from Part II, assembled.

Measuring It: nsys and ncu

```
# Nsight Systems: whole-app timeline
nsys profile --stats=true -t cuda,nvtx \
  -o report ./add_parallel.py

# Nsight Compute: per-kernel counters
ncu --section SpeedOfLight \
  --metrics sm_warps_active.avg.\
  pct_of_peak_sustained_active,gpu_time_duration.avg \
  --target-processes all \
  --print-summary per-gpu -o report ./add_parallel.py
```

- **nsys** answers “**where does time go** — is the GPU starved or blocked?”
- **ncu** answers “**why is this kernel slow** — occupancy? stalls? cache misses?”
- Run **both**: nsys alone misses in-kernel inefficiency; ncu alone can't see whether kernels are **fed data fast enough**.
- That long `sm_warps_active...` metric **is** achieved occupancy.

Plain English: nsys is the stopwatch for the whole machine; ncu is the microscope on one kernel.

Parallelism Cuts Runtime 22× at Only 38.7% Occupancy

Metric	Sequential	Parallel
Kernel time (ms)	48.21	2.17
GPU utilization	1.5%	95%
Achieved occupancy	1.3%	38.7%
Warp exec. efficiency	3.1%	100%

Table 6-6 (illustrative — real benchmarks in the book's repo^[7]). Warp efficiency 3.1% $\approx 1/32$: one active lane per warp.

- Note: 38.7% occupancy already bought 22× — you don't need 100%.
- GPU utilization, SM active %, achieved occupancy, and warp efficiency are **related but different metrics** — which is exactly how **95%**, **38.7%**, and **100%** coexist on one kernel.

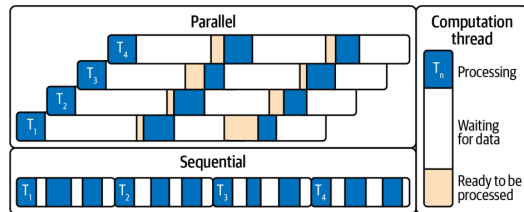


Figure 6-20. Sequential timeline has idle gaps; in the parallel version other warps' work fills them

A Busy GPU Can Still Be Waiting on Memory (LLM Decode)

95% utilization at only 38.7% occupancy, already 22× faster — so we don't need 100%. But are we at peak FLOPS? No: vector add mostly moves data.

- After parallelism, the next lever is per-warp efficiency (ILP etc., Chapter 8). But: **even at 100% occupancy**, performance suffers if the kernel is **memory bound** — limited by data movement, not compute.
- The book's canonical example: the **decode phase of an LLM** — every generated token streams **model weights** from HBM into registers/shared memory.
- Hundreds of billions of parameters $\times \sim 1$ byte $\approx$ **hundreds of GB** per pass — this **saturates memory bandwidth** regardless of thread count.

The trend that makes this worse (book note)

GPU FLOPS are outpacing memory bandwidth. Blackwell HBM3e delivers ~ 8 TB/s, but compute and model sizes grow faster — **optimizing memory movement is absolutely critical** in modern AI workloads.

Plain English: when the whole job is “carry boxes from the warehouse,” hiring more clerks at the desk doesn't help — that's what the roofline model (Part V) formalizes.

__launch_bounds__: Compile-Time Occupancy Control

```
// promise: never launched with > 256 thr/block  
// request: keep >= 16 blocks resident per SM  
__global__ __launch_bounds__(256, 16)  
void myKernel(...) { /* ... */ }
```

- $16 \times 256 = 4,096 > 2,048$ limit $\Rightarrow$ compiler **clamps** to 8 blocks and warns:
ptxas warning:minnctapersm will be ignored

- Effect: compiler **caps per-thread registers** (and may restrict unrolling/inlining) to avoid spills and fit **more warps per SM**.
- The deal: trade a bit of **per-thread performance** for **more consistent warp throughput** (more warps in flight).
- Danger zone: forcing too many threads $\Rightarrow$ registers run out $\Rightarrow$ **spilling to local memory** — slower than what you started with.

Plain English: you're trading each worker's toolbox size for the number of workers on the floor — the sweet spot is empirical.

The Occupancy API: Runtime Autotuning

```
int minGridSize = 0, bestBlockSize = 0;
// if kernel uses extern __shared__, set truthfully:
size_t dynSmemBytes = 0;
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &bestBlockSize, myKernel,
    dynSmemBytes, /*blockSizeLimit=*/0);
// cover N, but don't go below the min grid
// that saturates occupancy:
int gridSize = std::max(minGridSize,
    (N + bestBlockSize - 1) / bestBlockSize);
myKernel<<<gridSize, bestBlockSize,
    dynSmemBytes>>>(...);
```

- Computes the block size that **maximizes occupancy** given the kernel's actual register/shared-memory usage.
- Two gotchas (book):
minGridSize saturates occupancy — it is **not** the grid that covers N (take the max); and **pass real dynamic shared-memory bytes**.
- Book tip: **validate with $\pm 1-2$ candidate block sizes** — register pressure and L2 behavior can make slightly **sub-maximal occupancy faster**.